A Performance Evaluation of Hash Functions for IP Reputation Lookup Using Bloom Filters

Conference Paper · August 2015

DOI: 10.1109/ARES.2015.101

4 authors, including:

Hugo Francisco Gonzalez
Universidad Politécnica de San Luis Potosí
26 PUBLICATIONS **715** CITATIONS
SEE PROFILE

Natalia Stakhanova
University of New Brunswick
63 PUBLICATIONS **1,883** CITATIONS
SEE PROFILE

Ali A. Ghorbani
University of New Brunswick
344 PUBLICATIONS **20,897** CITATIONS
SEE PROFILE

A Performance Evaluation of Hash Functions for IP Reputation Lookup using Bloom Filters.

Marc Antoine Gosselin-Lavigne, Hugo Gonzalez, Natalia Stakhanova, Ali A. Ghorbani
Information Security Center of Excellence
Faculty of Computer Science, University of New Brunswick
{ma.gl,hugo,natalia,ghorbani}@unb.ca

***Abstract*—IP reputation lookup is one of the traditional methods for recognition of blacklisted IPs, i.e., IP addresses known to be sources of spam and malware-related threats. Its use however has been rapidly increasing beyond its traditional domain reaching various IP filtering tasks. One of the solutions able to provide a necessary scalability is a Bloom filter. Efficient in memory consumption, Bloom filters provide a fast membership check, allowing to confirm a presence of set elements in a data structure with a constant false positive probability. With the increased usage of IP reputation check and an increasing adoption of IPv6 protocol, Bloom filters quickly gained popularity. In spite of their wide application, the question of what hash functions to use in practice remains open.**

In this work, we investigate a 10 cryptographic and non-cryptographic functions for on their suitability for Bloom filter analysis for IP reputation lookup. Experiments are performed with controlled, randomly generated IP addresses as well as a real dataset containing blacklisted IP addresses. Based on our results we recommend two hash functions for their performance and acceptably low false positive rate.

***Keywords*-Theory; complexity measusres; performance measures**

I. Introduction

The unprecedented growth of spam and malware-related threats on the Internet has reached epidemic proportions. Aggravated by the increasing use of botnets allowing to recycle compromised end hosts in large amounts, the situation has forced many organization to employ an aggressive reputation filtering to complement a first line of defense.

Traditionally, reputation filtering has been one of the mechanisms to detect IP addresses known to be a source of spam and email-born threats often referred to as *DNS blacklisting* or *IP blacklisting*. This mechanism was widely employed by SMTP servers to identify and filter spam emails without expensive content-based analysis. With a rapid increase in the amount of 'bad' IP addresses, reputation services have expanded to include *white lists* containing IP addresses with an impeccable history and *gray lists* maintaining information on a new and fast growing category of IP addresses with no clear verdict. In addition to rapidly expanding storage requirements, reputation filtering is being increasingly used for various IP filtering tasks at the network edge requiring faster and reliable implementation. The increased use of IPv6 poses additional scalability concerns requiring to perform IP matching on prefixes of varying length (8 to 32 bits for IPv4 and 16 to 64 bits for IPv6). The challenge is to implement a scalable solution able to accommodate large IP lookup tables while providing high accuracy.

One of the lookup schemes that has recently received a significant attention in networking research community is a Bloom filter [6]. A Bloom filter a memory-efficient data structure that enables fast membership checks. The main features that make a Bloom filter attractive are its constant access time and its ability to compactness. The compactness is achieved through the use of hash functions that allow to create a unique 'signature' for each of the stored items. The use of hash functions unfortunately brings a possibility of false positives that comes from hash collisions, i.e., cases when different items result in the same 'signature'. False positive rate can be generally minimized through a selection of a collisions-resistant hash function or an application of multiple hash functions. Both solutions however are computationally intensive and increase latency in an item lookup. To resolve some of these problems a number of research studies have focused on Bloom filter's improvement introducing the weighted Bloom filter [9], the scalable Bloom filter [4] and the Bloomier filter [10]. The choice of hash functions however remains to be critical.

In this paper, we investigate a performance of 10 cryptographic and non-cryptographic hash functions on their suitability for IP Reputation Lookup using Bloom filters. With the latest increased interest in Bloom filters in networking applications we specifically focus on both IPv4 and IPv6 addresses, both in their integer and string representations. We judge the quality of the hash functions based on functions' performance and the false positive rate of the resulting Bloom filter. Based on the conducted experiments we recommend the use of non-cryptographic hash functions, especially Hsieh hash.

The paper is organized as follows. Section II presents related work. Section III describes the background and the selected hash functions. Evaluation results are presented in Section IV and finally Section VI concludes with the paper.

II. Related work

A Bloom filter was originally introduced by Bloom [6] for general applications requiring error-free performance; and has quickly become popular in databases and for dictionary storage. Nowadays, Bloom filters can be found

in Google Chrome, Bitcoin, Squid web proxy, and SPIN model checker.

In the recent years a Bloom filter has seen an increased interest in a networking domain and security. One of the first uses of Bloom filters for security purposes traces back to Spafford's work on storing dictionary of weak passwords [34]. More recently Bloom filters found application in intrusion detection [13], [19], virus and spam detection [35], [28], access control [16] and IP traceback [33].

To address the limitations of the basic Bloom filter (i.e., possibility of false positives due to hash collisions and computational intensity), numerous variations have been proposed for a variety of uses including the counting Bloom filter [15], the weighted Bloom filter [9], the Scalable Bloom filter [4] and the Bloomier filter [10]. There has also been several efforts in improving the speed and accuracy of Bloom filters, which included partitioned hashing [18] and the development of techniques allowing use of small number of hash functions without a significant increase in the false positive rate.

In all these studies selection of hash functions plays a critical role effecting an accuracy, effectiveness and scalability of the solution. There have been very little guidance from a research community on a suitability of various hash functions for different applications. Several studies focused on analysis of hashing functions' performance specifically for hardware applications. Ramakrishna et al [31] defined a general class of functions H_3 that can be easily implemented in hardware, concluding that any function picked at random from this class would be suitable for fast hardware applications. Hua et al. [22] explored the performance of CRC-32, H3, MD5 and others. The authors introduced a new hardware hash function pointing out that existing non-cryptographic hardware hash functions are generally low quality and do not scale to large bus sizes, while high quality software hash functions are simply too computationally intensive for hardware level implementation.

Among the studies that investigated a behavior of hash functions for software applications are a number of studies focusing on hash function specifically for hash-based packet selection, i.e., an approach that aims to provide a truly random network packet sampling through hashing of packet content. Molina et al [3] focused on four hash functions for a packet sampling concluding that Lookup and CRC32 functions performed the best. Henke et al [20] analyzed a broad spectrum of hash functions recommending BOB and OAAT as the best in performance. In both cases hash functions were considered in a context packet attributes and potential influence of hash function selection on sampling. A general comparison of cryptographic functions' performance has been conducted by Dai [12]. As been noted by other studies MD5 and SHA are uniformly considered to be the best in performance. While it included perhaps the largest selection of hash functions, the cryptographic functions themselves are often prohibitively expensive for the majority of software applications.

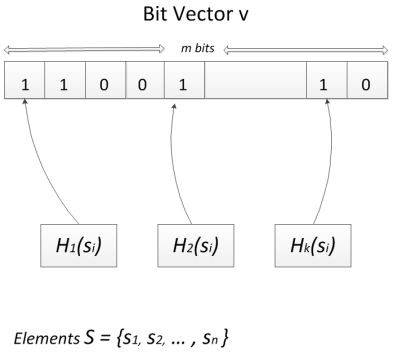

Figure 1. An illustration of a Bloom filter

In this work we look at a more general case of hash functions' use for software applications requiring IP lookup, we include both cryptographic and non-cryptographic functions and consider both IPv4 and IPv6 addresses.

III. Background

A. Bloom filter

A Bloom filter is a simple data structure that allows space-efficient representation of set elements to support fast memberships queries. Given a set $S = s_1, s_2, \ldots, s_n$ of n elements, the Bloom filter relies on a bit vector and a set of hash functions. The idea of a bloom filter is illustrated in Figure 1.

Given a vector v of m bits, initially set to 0, and a set of k hash functions $H = h_1, h_2, \ldots, h_k$, an element $s_i \in S$ will set all bits of vector v corresponding to the following positions $h(s_i)_1, h(s_i)_2, \ldots, h(s_i)_k$ to 1. Note that the same bit might be set to 1 multiple times either through different hash functions for the same element s_i or different elements of S. The process of verifying if an element b is present in the Bloom filter is analogous to the insertion process with one difference: instead of setting the bits corresponding to the hashes to 1 we verify if the bits corresponding to positions $h(b)_1, h(b)_2, \ldots, h(b)_k$ are already set to 1. If at least one bit is set to 0 then an element b is not in the Bloom filter.

While the Bloom filter has many advantageous features, such as a relatively small size (at most a few bytes per element) and a fast access time it does come with an obvious drawback: a possibility of false positives on membership checks. A false positive occurs when the hashes from an element not in the Bloom filter overlaps with a combination of hashes from elements that are in the Bloom filter. Given a size m of a bit vector, a number of elements n present in a Bloom filter, and k hash functions, a probability of a false positive P_{fp} is given as follows:

$$P_{fp} = (1 - (1 - \frac{1}{m})^{kn})^k \quad (1)$$

This classic formula for estimating false positive rate has recently been called into question [7], [11]. The

Table I
EMPLOYED HASH FUNCTIONS

Hash	Digest size	Seed or salt
City Hash (64 bit) [30]	64 bits	seed
City Hash (64 bit, long) [30]	2 × 32 bits	seed
City Hash (32 bit) [30]	32 bits	salt
CRC32 [8]	32 bits	salt
Hsieh Hash [21]	32 bits	salt
Lookup3 [24]	32 bits	seed
Murmur3 [5]	32 bits	seed
One at a Time [23]	32 bits	salt
RS Hash [29]	32 bits	salt
Sbox [27]	32 bits	salt
Spooky [25]	32 bits	seed
md5 [32]	128 bits	salt
md5 (long) [32]	4 × 32 bits	n/a

updated accuracy analysis that gives a stricter bound on the false positive rate[1] is as follows:

$$P_{fp} = \frac{m!}{m^{k(n+1)}} \sum_{i=1}^{m} \sum_{j=1}^{i} (-1)^{i-j} \frac{j^{kn} i^k}{(m-i)!j!(i-j)!} \quad (2)$$

B. Hash functions

We investigated a collection of ten hash functions on their suitability for Bloom filter analysis. An overview of the collection is given in Table I. Among them were nine non-cryptographic hash functions, i.e., did not implement any security features and one cryptographic function. The functions were selected based on the results of the previous analysis of hash functions for their applicability in various domains. As such, a study on hash-based network traffic attribute selection conducted by Henke et al [20] suggested the following hash functions: Hsieh (also known as SuperFastHash), Lookup, One at a time, sbox and RS. A more general study of non-cryptographic hash functions by Estébanez et al. recommended Murmur function along with Hsieh and Lookup [14]. Similar recommendation came from [3]. Two additional functions: City and Spooky (used internally by Google [30]) although have not be subject of any study, were developed by R. Jenkins, who also designed two other functions that proved to be efficient: One at a time and Lookup [25]. In all cases the latest versions of hash functions were selected. For example, we tested the latest Murmur3 out of MurmurHash family hash functions instead for its robustness and speed.

Finally, we used MD5 as a representative cryptographic hash function. Cryptographic hash functions are more secure and in general are more resource-consuming than non-cryptographic functions. Since the focus of this study is not on function's security properties, the comprehensive analysis of various cryptographic hash functions is beyond the scope of this work. The non-cryptographic functions, on the other hand, have shown to be vulnerable in certain adversarial settings (i.e., causing a disproportional packet selection [17]). However, the presence of adversary in our setting does not introduce a bias into membership check results and thus a study of non-cryptographic hash functions for a purpose of IP reputation checking is sufficient.

In general hash functions vary in length of their output value, i.e., digest. Whenever possible we tested functions that provided a 32 bit digest. When this was not possible, the values were trimmed to 32 bits by using several subblocks of 32 bits. For the study purpose, the use of 32 bit keys for a Bloom filter was acceptable as it allowed to build a sufficiently large Bloom filter, i.e., allowing to accommodate an excess of 500 million elements in a bit vector.

IV. EVALUATION

We investigate the performance of the hash functions in two settings: with a controlled set of randomly generated IP addresses and with a set blacklisted IP addresses provided by third-party companies.

A. Controlled dataset

In this experiment, we focused on the evaluation of hash functions' performance for both the integer and the string representation of IPv4 and IPv6 addresses.

Two datasets were created containing one million unique IP addresses randomly generated using Intel's RdRand method [26]. The experiments were performed for integer representations of IPv4 and IPv6 (32 bit and 128 bit unsigned integer respectively) and string representations of IPv4 and IPv6 addresses (ASCII string).

The hash functions were evaluated according to two metrics: *performance*, i.e., the time necessary to hash IP addresses, and a *false positive rate* of a Bloom filter, i.e., the percentage of IP addresses that are falsely detected as present in a Bloom filter.

When computing time we actually compute the time required to obtain four 32 bit keys. To do this we seed the hash function or, if this is not possible, salt the IPs. To test the false positive rate we build a Bloom filter that has a length of 800,000 bits and we insert 100,000 unique IPs into it. We then query the Bloom filter using 900000 unique IPs that where not inserted into the Bloom filter. Any of those IPs that are said to be in the Bloom filter will be false positives. This is done with two independent datasets and the results are combined. The experiments were run on Intel i7, 16 Gb RAM.

Results: The evaluation results are presented in Tables II, IV, III and V.

The most accurate hash function for IPv4 address represented as integers is the Lookup3 hash. This function is 1.7 times slower than the fastest hash function for such representation, RS hash. However, RS hash is unusable as it is by far the least accurate hash function. Hsieh appears to be the best all-around hash as it is less than 1.1 times slower than RS while suffering less than a 1% increase in false positive over Lookup3.

Staying with integer representations, but this time for IPv6 addresses, we find that Hsieh has the fastest average

[1] A detailed analysis can be found in [11]

Table II
EVALUATION RESULTS FOR INTEGER REPRESENTATION OF IPV4 ADDRESSES (CONTROLLED DATASET)

Hash	Time per query (ns)	FP %	FP count
City Hash (32 bit)	51.9	2.39	42991
City Hash (64 bit)	99.0	2.40	43231
City Hash (64 bit, long)	51.7	2.40	43190
Hsieh Hash	16.4	2.40	43217
Lookup3	26.1	**2.38**	**42840**
Murmur3	31.3	2.41	43304
One at a Time	39.9	2.43	43710
RS Hash	**15.1**	2.92	52632
Sbox	90.5	2.41	43440
Spooky	109.3	2.40	43203
crc32	19.6	2.40	43352
md5	697.3	**2.38**	42850
md5 (long)	177.2	2.40	43281

Table III
EVALUATION RESULTS FOR INTEGER REPRESENTATION OF IPV6 ADDRESSES (CONTROLLED DATASET)

Hash	Time per query (ns)	FP %	FP count
City Hash (32 bit)	168.5	2.40	43152
City Hash (64 bit)	117.7	2.39	43093
City Hash (64 bit, long)	62.3	2.39	43057
CRC32	72.3	2.41	43352
Hsieh Hash	**33.2**	2.41	43291
Lookup3	56.6	2.39	43088
Murmur3	50.7	2.39	43093
One at a Time	85.4	2.39	43031
RS Hash	43.6	2.94	52961
Sbox	137.9	**2.38**	**42857**
Spooky	214.7	2.39	43056
md5	695.3	2.41	43332
md5 (long)	177.2	2.41	43297

Table IV
EVALUATION RESULTS FOR STRING REPRESENTATION OF IPV4 ADDRESSES (CONTROLLED DATASET)

Hash	Time per query (ns)	FP %	FP count
City Hash (32 bit)	152.7	2.42	43604
City Hash (64 bit)	119.7	2.40	43126
City Hash (64 bit, long)	62.4	2.40	43194
CRC32	58.4	2.40	43116
Hsieh Hash	**32.4**	2.41	43420
Lookup3	53.3	2.39	42980
Murmur3	57.5	2.40	43278
One at a Time	72.1	2.40	43251
RS Hash	38.6	3.18	57164
Sbox	129.3	**2.38**	**42857**
Spooky	131.9	2.41	43353
md5	697.7	**2.38**	42872
md5 (long)	180.8	2.39	43035

Table V
EVALUATION RESULTS FOR STRING REPRESENTATION OF IPV6 ADDRESSES (CONTROLLED DATASET)

Hash	Time per query (ns)	FP %	FP count
City Hash (32 bit)	266.4	2.39	43066
City Hash (64 bit)	255.6	2.39	43000
City Hash (64 bit, long)	130.7	2.40	43159
CRC32	197.9	2.39	43109
Hsieh Hash	**63.8**	2.40	43258
Lookup3	118.0	2.39	43106
Murmur3	112.3	2.39	42990
One at a Time	224.1	2.40	43262
Sbox	237.8	2.38	42820
Spooky	244.6	2.41	43463
RS Hash	115.7	3.59	64676
md5	701.2	2.39	42976
md5 (long)	181.8	**2.37**	**42603**

time while sbox has the lowest false positive count. Sbox is sadly more than four times slower than Hsieh, so it is discarded. While Hsieh does have a decent false positive count (roughly a 1% increase in false positives over sbox), City Hash (64 bit, long), Murmur3 and Lookup3 all have a lower false positive count without having a dramatic hit in terms of speed. Any of those hashes would be appropriate for a 128 bit integers.

For IPv4 addresses that are represented as strings, two hashes stand out. Hsieh hash is, once again, the fastest while only having about a 1% increase in false positives over the hash with the lowest amount of false positive (sbox). Lookup3, meanwhile, is somewhat slower than Hsieh but makes up for it by being more accurate. Both hashes are a good choice.

For IPv6 addresses represented as strings, Hsieh is the fastest while MD5 (long) is the most accurate. Hsieh has 1.5% more false positives than MD5 (long) but it is nearly three times faster. Murmur3 is somewhere in between Hsieh and MD5, being 1.8 times slower than Hsieh but having only 0.9% more false positives than MD5 (long). MD5 (long) is a little bit on the slow side and Hsieh and Murmur3 would appear to be the better choices.

Hsieh is the most consistently good hash function across our various tests in terms of balancing speed and a low false positive rate, despite the fact that it does tend to produce a higher false positive count than most of the other hashes we have tested. That being said, with the exception of RS hash none of the hash functions have a false positive count that is worse than a 2% increase over the most accurate hash function while big variations are observed in terms of processing speed.

Unsurprisingly, the integer representation of IP addresses is processed faster than the string representation by all the hash functions with the exception of MD5, which has a roughly equivalent processing time for all tests. The string representation seems to do a little better overall when it comes to the false positive rate, but much of the difference appears to come from MD5 producing a lower false positive rate when processing string representations

of the IP addresses.

Table VI
EVALUATION RESULTS FOR REAL DATASET

Hash	FP %	FP count
City Hash (32 bit)	2.41	21720
City Hash (64 bit)	2.40	21617
City Hash (64 bit, long)	2.43	21857
CRC32	2.42	21743
Hsieh Hash	2.41	21687
Lookup3	**2.37**	21323
Murmur3	2.40	21626
One at a Time	2.39	21466
Sbox	2.41	21818
Spooky	2.39	21538
RS Hash	3.00	27030
md5	2.41	21673
md5 (long)	2.42	21784

B. Real-data set

Since the controlled dataset might be representing an ideal scenario, we employ a more realistic set of IP addresses that reflects the actual (biased) distribution of blacklisted IP addresses. This dataset was built with IP addresses from two different sources:

- One million IPv4 addresses were donated by Ara Labs [1] from their intelligence services. All IP addresses were blacklisted as during September, 2013.
- 80 thousand IPv4 addresses were obtained from the I-Blocklist [2] from the following lists: adds, spyware, proxy, badpeers, hijacked, dshield, webexploit, drop, zeus, spyeye, palevo, malicious, malcode, addservers. Since the lists provides domain information as well, for the final dataset only IP addresses were retained.

The final dataset was verified to be free of duplicates and invalid IP addresses (e.g., localhost, 0.0.0.0). The experiments were performed in the same fashion as for randomly generated data, and only for string representation for IP addresses. The results of experiments with this dataset are presented in Table VI.

As the results show, although performance slightly deviates from the one with the controlled set, the worst performing (RS hash 3% FP) and the best performing functions (Lookup3 2.37% FP rate) remain the same.

V. DISCUSSION

With the exception of RS hash every hash function that was tested produced Bloom filters that had similar false positive rates. This shows that when selecting a hash function for a Bloom filter speed should be the main factor as most good hash functions will produce similar false positive rates. Of the tested hash functions, Hsieh performed across all tests. Lookup3 and Murmur3 both also perform well, especially for IPv4 strings and IPv6 strings respectively. Integer representations of IP addresses perform better in terms of speed than string representations, so they should be used when possible.

Ultimately, if a lower false positive is required increasing the size of the Bloom filter would be necessary as the hash functions we tested all have similar false positives.

While MD5 achieves good false positive rates it is much slower than non-cryptographic hashes without providing any tangible benefits and it should thus be avoided.

Generally, based on theoretical analysis the amount of false positives is not attributed to a particular hash function, but rather characteristics of a Bloom filter (e.g., its size, number of hash functions employed). Thus we also consider false positives. Overall, the false positive rate obtained in our experiments matches the predicted performance. The expected FP rate with the classic formula (1) is 2.396869%, while with the updated formula (2) is 2.396876%. In our experiments, the fraction of false positives in most cases ranged from 2.37% to 2.41% with RS hash significantly deviating from the majority with FP rate reaching 3.59% (string representation, IPv6). It should be noted though that this performance of RS hash is compensated by speed. RS hash is one of the fastest functions tested in this experiment. We see potential application that can tolerate this inaccuracy for faster performance.

This work has some limitations. Tests where only performed on one architecture and no hardware based hash functions where tested. Some hash functions, notably City Hash, may perform better on a different architecture. The testing assumes a Bloom filter that requires indices no larger than 32 bit. While the results might be different for Bloom filters with larger indices we do not believe that such large Bloom filter are frequently utilized. As a corollary we might find different results if we only require a much smaller index, e.g. 8 or 16 bits. Additionally, our work only tests using IP addresses as keys. It is possible that hash functions such as MD5 would perform better when the keys are larger.

VI. CONCLUSION

We found that non-cryptographic hash functions are appropriate when implementing a Bloom filter that stores IP addresses. While Hsieh hash provided the fastest lookup time, Lookup3 and closely performing Murmur3 showed the best overall performance.

The rest of the tested non-cryptographic hash functions (with the exception of RS hash) showed satisfactory false positive rates. There was very little difference in false positive between every hash (RS hash notwithstanding), such that any change in false positive can easily be erased by minutely increasing the size of the Bloom filter. This appeared to indicate that, in practice, the fastest "good" hash function available for a specific architecture should be used when implementing a Bloom filter.

REFERENCES

[1] "Ara labs,". http://www.aralabs.com/.

[2] "I-Blocklist,". https://www.iblocklist.com/lists.php.

[3] M. Molina A, S. Niccolini B, and N. G. Duffield C, "A comparative experimental study of hash functions applied to packet sampling," in *In Proc of International Teletraffic Congress (ITC)*, 2005.

[4] Paulo Sérgio Almeida, Carlos Baquero, Nuno Preguiça, and David Hutchison, "Scalable bloom filters," *Inf. Process. Lett.*, vol. 101, pp. 255–261, March 2007.

[5] Austin Appleby. "smhasher,", 2008.

[6] Burton H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, pp. 422–426, 1970.

[7] Prosenjit Bose, Hua Guo, Evangelos Kranakis, Anil Maheshwari, Pat Morin, Jason Morrison, Michiel Smid, and Yihui Tang, "On the false-positive rate of bloom filters," *Inf. Process. Lett.*, vol. 108, pp. 210–213, October 2008.

[8] Gary S. Brown. "Crc32 c implementation,", 1986.

[9] J. Bruck, Jie Gao, and Anxiao Jiang, "Weighted bloom filter," in *Information Theory, 2006 IEEE International Symposium on*, pp. 2304–2308, 2006.

[10] Bernard Chazelle, Joe Kilian, Ronitt Rubinfeld, and Ayellet Tal, "The bloomier filter: an efficient data structure for static support lookup tables," in *Proceedings of the fifteenth annual ACM-SIAM symposium on Discrete algorithms*, SODA '04, pp. 30–39, Philadelphia, PA, USA, 2004. Society for Industrial and Applied Mathematics.

[11] Ken Christensen, Allen Roginsky, and Miguel Jimeno, "A new analysis of the false positive rate of a bloom filter," *Inf. Process. Lett.*, vol. 110, pp. 944–949, October 2010.

[12] Wei Dai. "Crypto++ 5.6.0 benchmarks,". http://www.cryptopp.com/benchmarks.html, 2009.

[13] Sarang Dharmapurikar and J.W. Lockwood, "Fast and scalable pattern matching for network intrusion detection systems," *Selected Areas in Communications, IEEE Journal on*, vol. 24, no. 10, pp. 1781–1792, 2006.

[14] César Estébanez, Yago Saez, Gustavo Recio, and Pedro Isasi, "Performance of the most common non-cryptographic hash functions," *Software: Practice and Experience*, 2013.

[15] Li Fan, Pei Cao, Jussara Almeida, and Andrei Z. Broder, "Summary cache: A scalable wide-area web cache sharing protocol," *IEEE/ACM Trans. Netw.*, vol. 8, pp. 281–293, June 2000.

[16] S. N. Foley and G. Navarro-Arribas, "A bloom filter based model for decentralized authorization," *Int. J. Intell. Syst.*, vol. 28, p. 565582, 2013.

[17] Sharon Goldberg, David Xiao, Eran Tromer, Boaz Barak, and Jennifer Rexford, "Path-quality monitoring in the presence of adversaries," *SIGMETRICS Perform. Eval. Rev.*, vol. 36, pp. 193–204, June 2008.

[18] Fang Hao, Murali Kodialam, and T. V. Lakshman, "Building high accuracy bloom filters using partitioned hashing," in *Proceedings of the 2007 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*, SIGMETRICS '07, pp. 277–288, New York, NY, USA, 2007. ACM.

[19] Byeong hee Roh, Ju Wan Kim, Ki-Yeol Ryu, and Jea-Tek Ryu, "A whitelist-based countermeasure scheme using a bloom filter against {SIP} flooding attacks," *Computers & Security*, vol. 37, no. 0, pp. 46 – 61, 2013.

[20] Christian Henke, Carsten Schmoll, and Tanja Zseby, "Empirical evaluation of hash functions for multipoint measurements," *SIGCOMM Comput. Commun. Rev.*, vol. 38, pp. 39–50, July 2008.

[21] Paul Hsieh. "Hash functions,", 2004.

[22] Nan Hua, Eric Norige, Sailesh Kumar, and Bill Lynch, "Non-crypto hardware hash functions for high performance networking asics," in *Proceedings of the 2011 ACM/IEEE Seventh Symposium on Architectures for Networking and Communications Systems*, ANCS '11, pp. 156–166, Washington, DC, USA, 2011. IEEE Computer Society.

[23] Bob Jenkins. "Hash functions,", 1997.

[24] Bob Jenkins. "Lookup3,", 2006.

[25] Bob Jenkins. "Spookyhash: a 128-bit noncryptographic hash,", 2012.

[26] John Mechalas. "User manual for the rdrand library (windows* version),", 2012.

[27] Bret Mulvey. "Hash functions,", 2007.

[28] Arulanand Natarajan, S. Subramanian, and K. Premalatha, "A comparative study of cuckoo search and bat algorithm for bloom filter optimisation in spam filtering," *Int. J. Bio-Inspired Comput.*, vol. 4, pp. 89–99, June 2012.

[29] Arash Partow. "General purpose hash function algorithms,", 2010.

[30] Geoff Pike and Jyrki Alakuijala. "The cityhash family of hash functions,", 2011.

[31] M.V. Ramakrishna, E. Fu, and E. Bahcekapili, "A performance study of hashing functions for hardware applications," in *In Proc. of Int. Conf. on Computing and Information*, pp. 1621–1636, 1994.

[32] Ronald Rivest. "Md5 c implementation,", 1990.

[33] Alex C. Snoeren, Craig Partridge, Luis A. Sanchez, Christine E. Jones, Fabrice Tchakountio, Stephen T. Kent, and W. Timothy Strayer, "Hash-based ip traceback," *SIGCOMM Comput. Commun. Rev.*, vol. 31, pp. 3–14, August 2001.

[34] Eugene H. Spafford, "Opus: preventing weak password choices," *Comput. Secur.*, vol. 11, pp. 273–278, May 1992.

[35] DineshC. Suresh, Zhi Guo, Betul Buyukkurt, and WalidA. Najjar. "Automatic compilation framework for bloom filter based intrusion detection,". in *Reconfigurable Computing: Architectures and Applications* (Koen Bertels, JooM.P. Cardoso, and Stamatis Vassiliadis, eds.), vol. 3985 of *Lecture Notes in Computer Science*, pp. 413–418. Springer Berlin Heidelberg, 2006.